

ENGR1050: Lecture 14

Differential equations and numerical methods

October 14, 2026



Agenda

- Last class we built a class for solving ODEs numerically together
- Today we'll get some extra practice using it to model a pendulum
- We have lab next Monday (**meet in GM lab**) where we will use *potentiometers* to measure angles of pendulums

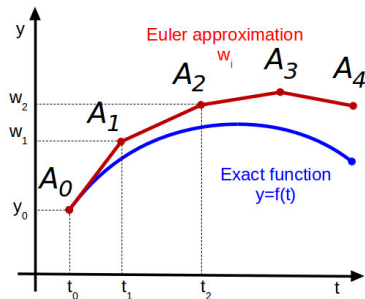
Numerical solution of ODEs

Many ODEs cannot be solved analytically, so we use numerical methods to approximate solutions. The simplest is the Explicit Euler method.

Definition

For the IVP $y'(t) = f(t, y)$ with $y(t_0) = y_0$ and step size $h > 0$, starting from t_0, y_0 , the Explicit Euler scheme updates via

$$t_{n+1} = t_n + h, \quad y_{n+1} = y_n + h f(t_n, y_n),$$



Last lecture's ODE_solver class

```
class ODE_solver:
    def __init__(self, t0, tf, y0, N):
        self.y0 = y0 # initial condition as numpy array
        self.t0 = t0 # initial time as float
        self.tf = tf # final time as float
        self.N = N # number of steps as integer
        self.h = (tf - t0) / float(N) # timestep size

        # We need to check what size y0 is and store that information
        self.dim = len(y0) # assumes that y0 is a list or array

        # Create a place to store the solution
        self.t_values = np.array([t0 + i*self.h for i in range(N+1)]) # time values
        self.solution = np.zeros((N+1, self.dim)) # solution array
        self.solution[0, :] = y0 # set initial condition

    def f(self, t, y):
        # Define the right-hand side of the ODE here, assuming a scalar t and a numpy array y
        # Example: dy/dt = -2y (for scalar y),
        # this function should return a numpy array of the same shape as y
        f = np.zeros_like(y)
        f[0] = -2 * y[0] # Example for the first component
        return f

    def solve(self):
        # Implement explicit euler: y_{n+1} = y_n + h*f(t_n, y_n)
        for n in range(self.N):
            t_n = self.t_values[n]
            y_n = self.solution[n, :]
            f_n = self.f(t_n, y_n)
            self.solution[n+1, :] = y_n + self.h * f_n
        return self.solution
```

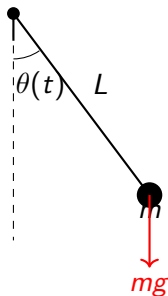
We discussed: best practices for solving an ODE.

- Implement the RHS of the ODE in the f method
- Instantiate the class with initial conditions, final time, and number of steps
- Call the `solve()` method to compute the numerical solution
- **Convergence study:** Increase N and run again by re-instantiating the class; repeat until solution doesn't change
- **Validation:** Compare to analytical solution or asymptotic behavior

The homework consists of solving two different numerical ODEs with this code. We will work through an example today that should help you finish the homework and also prepare you for the upcoming lab.

Derivation: nonlinear pendulum equation

Newton: acceleration in tangent direction a_t :



$$ma_t = -mg \sin \theta \quad (1)$$

$$mL\theta'' = -mg \sin \theta \quad (2)$$

$$\theta'' = -\frac{g}{L} \sin \theta \quad (3)$$

Where:

- $a_t = L\theta''$ (tangential acceleration)
- Restoring force: $-mg \sin \theta$
- Tension T acts radially (canceled out)

Result: $\theta'' + \frac{g}{L} \sin(\theta) = 0$ (nonlinear due to $\sin \theta$)

Note for small angle $\sin(\theta) \sim \theta$: $\theta'' + \frac{g}{L}\theta = 0$ and we recover the harmonic oscillator.

Example: nonlinear pendulum

- The nonlinear pendulum is governed by the ODE

$$\theta''(t) + \frac{g}{L} \sin(\theta(t)) = 0,$$

where $\theta(t)$ is the angle of the pendulum from vertical, g is the acceleration due to gravity, and L is the length of the pendulum.

- **Our code assume first-order ODEs:** We must convert it to a system of first-order ODEs by defining $\omega = \theta'$:

$$\begin{cases} \theta' = \omega, \\ \omega' = -\frac{g}{L} \sin(\theta). \end{cases}$$

- We can now implement this system in our ODE solver class.

What's coming

This unit's project: 1D helicopter model

Ingredient 1:

DC motor

Requires:

- Power source
- Line out to GND

Ingredient 2:

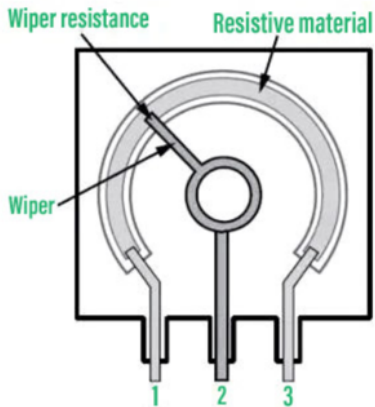
Potentiometer

Requires:

- Power source
- Line out to GND
- GPIO pin to make reading



Upcoming lab: learn how to use a potentiometer



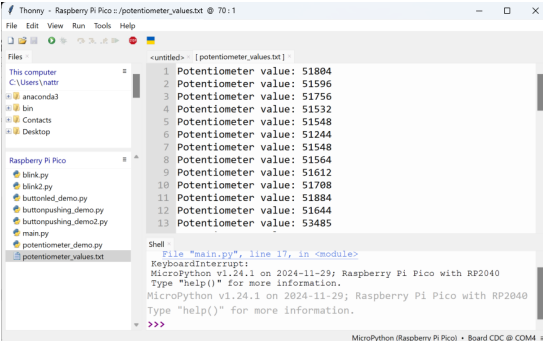
Reading a potentiometer with the picos

Goal: Read analog sensor data using the Raspberry Pi Pico's ADC (analog-to-digital converter)

```
from machine import ADC, Pin
import time

# Initialize ADC on pin 26 (for Raspberry
↳ Pi Pico)
potentiometer = ADC(Pin(26))

while True:
    # Read the ADC value (0-65535)
    pot_value = potentiometer.read_u16()
    print("Potentiometer value:",
    ↳ pot_value)
    time.sleep(0.1)
```



Thonny - Raspberry Pi Pico :: /potentiometer_values.txt @ 70:1

File Edit View Run Tools Help

Files

- This computer
- C:\Users\nattr
- anaconda3
- bin
- Contacts
- Desktop
- Raspberry Pi Pico
 - blink.py
 - blink2.py
 - buttonled_demo.py
 - buttonpushing_demo.py
 - buttonpushing_demo2.py
 - main.py
 - potentiometer_demo.py
 - potentiometer_values.txt

<untitled> [potentiometer_values.txt]

```
1 Potentiometer value: 51804
2 Potentiometer value: 51596
3 Potentiometer value: 51756
4 Potentiometer value: 51532
5 Potentiometer value: 51548
6 Potentiometer value: 51244
7 Potentiometer value: 51548
8 Potentiometer value: 51564
9 Potentiometer value: 51612
10 Potentiometer value: 51708
11 Potentiometer value: 51884
12 Potentiometer value: 51644
13 Potentiometer value: 53485
```

Shell

File "main.py", line 17, in <module>
KeyboardInterrupt:
MicroPython v1.24.1 on 2024-11-29; Raspberry Pi Pico with RP2040
Type "help()" for more information.
MicroPython v1.24.1 on 2024-11-29; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>

MicroPython (Raspberry Pi Pico) • Board CDC @ COM4

Getting data off of the pico and onto your laptop to read

Goal: Save potentiometer readings to a file on the Pico's filesystem so you can transfer it to your laptop later.

```
from machine import ADC, Pin
import time

# Initialize ADC on pin 26 (for Raspberry Pi Pico)
potentiometer = ADC(Pin(26))

# Open the file in write mode
with open("potentiometer_values.txt", "w") as file:
    while True:
        pot_value = potentiometer.read_u16() # Read the ADC value (0-65535)
        file.write(f"Potentiometer value: {pot_value}\n")
        file.flush() # Ensure the value is written to the file immediately
        print("Potentiometer value:", pot_value)
        time.sleep(0.1)
```

To transfer: In Thonny, View → Files → Right-click file → Download to computer

Today

- Step through the pendulum code together
- Finish homework together
- Get ready for Monday's lab
- Questions?